

Projet « Scribe » : Dossier de PRÉ-PRODUCTION

Assistant de réunion intelligent : captation, transcription, compte-rendu et suivi

1. Contexte

L'application que vous allez concevoir puis réaliser s'appelle « **Scribe** ». Elle a pour objectif d'assister les réunions de bout en bout : capter la parole à la source, la transcrire, identifier qui dit quoi, puis produire automatiquement un compte-rendu structuré accompagné d'une liste d'actions. L'application devra aussi reconnaître l'ambiance et les thèmes d'une réunion, et permettre à l'utilisateur de retrouver ses anciennes réunions dans un espace personnel.

La particularité de Scribe est de capter la réunion elle-même, dans deux situations d'usage opposées. Soit la réunion est à distance, et l'application héberge la visioconférence afin de récupérer l'audio directement. Soit la réunion est en présentiel, et l'application sert de dictaphone intelligent en captant le micro de l'appareil. Ces deux modes débouchent sur la même chaîne de traitement, mais posent des problèmes techniques différents : c'est précisément cette asymétrie qui fait l'intérêt du sujet.

À terme, cette application pourrait permettre à une équipe de suivre ses décisions, ses actions en cours et ses sujets récurrents dans le temps. Le défi ici n'est pas seulement technique : il s'agit aussi de penser une interface agréable, un usage fluide, et une expérience qui donne envie d'être réutilisée.

2. Objectifs

Vous préparez la réalisation d'une application web permettant :

- la **captation** de la parole via deux modes obligatoires : une **visioconférence intégrée** à l'application, et un **mode dictaphone** pour les réunions en présentiel ;
- la **transcription** automatique de la réunion ;
- l'**attribution des propos** aux différents intervenants (qui parle quand) ;
- la **classification** du ton et des thèmes de la réunion ;
- la **génération** d'un compte-rendu structuré et d'une liste d'actions ;
- la **consultation historique** des réunions dans un tableau de bord personnel.

La phase de pré-production vise à **sécuriser le développement** : toute décision fonctionnelle, technique, UX et organisationnelle doit être formalisée afin que la phase de Production se concentre sur l'implémentation.

3. Compétences évaluées

Cette phase est adossée au titre **RNCP 36146 : Concepteur développeur de solutions digitales** (niveau 6). Elle couvre principalement le bloc **BC01** et amorce le bloc **BC02**.

Réf.	Bloc / compétence visée
BC01	Spécifier la demande au regard des objectifs de la solution afin de produire le document de spécification des besoins
BC01	Réaliser un état de l'art des outils et approches existants afin de proposer la solution optimale (coût, faisabilité, délais, performance, pérennité)
BC01	Concevoir l'architecture logicielle et les spécifications fonctionnelles pour atteindre les objectifs définis
BC02	Choisir les outils et approches méthodologiques à mettre en œuvre en fonction des objectifs et contraintes du projet

4. Organisation du travail

- **Taille des groupes** : 4 étudiants (désignez un·e Scrum Master).
- **Méthode** : Scrum en sprints de 1 semaine ; outils : GitHub Projects ou Jira.
- **Continuité** : ce dossier est la première phase d'un projet fil rouge. Les décisions prises ici engagent directement la phase de Production. Tout choix non tranché en pré-production devient une dette à payer en production.

5. Système de paliers : une épreuve à plusieurs niveaux

Le projet est conçu pour que chaque groupe trouve un chemin de réussite à sa mesure. Chaque fonctionnalité se décline en **trois paliers** :

- le **socle** est le minimum attendu. Livré proprement, testé et documenté, il suffit à valider les blocs de compétences et à obtenir une note correcte ;
- la **cible** est l'attendu normal d'un troisième année. C'est elle qui construit la note au-dessus de la moyenne ;
- l'**avancé** n'est jamais nécessaire pour valider. Il départage les groupes et récompense l'ambition technique.

Les paliers sont **indépendants d'une fonctionnalité à l'autre**. Un groupe peut viser le socle sur la diarisation et l'avancé sur le tableau de bord. Personne n'est pénalisé d'avoir choisi le socle ; on valorise seulement ceux qui montent. La grille ci-dessous gouverne les deux phases du projet : en pré-production, elle oriente vos choix, votre benchmark et votre backlog ; en production, elle structure l'implémentation et la notation.

Brique	Socle (suffit à valider)	Cible (attendu)	Avancé (départage)
Captation audio	Les deux modes fonctionnent : dictaphone (micro navigateur) et visio intégrée via SDK. La visio peut se contenter d'un audio global.	Visio récupérant des pistes audio séparées par participant ; dictaphone robuste (coupures gérées, fichiers longs découpés).	Bot de réunion (type Recall.ai) sur visio externe, ou visio auto-hébergée (LiveKit, Jitsi) avec enregistrement propre et reprise sur incident.
Transcription (STT)	Whisper, texte brut. En visio, le découpage par piste donne déjà « qui parle » presque gratuitement ; en dictaphone, on assume un texte sans locuteurs.	Diarisation réelle en mode dictaphone (API dédiée ou Whisper + pyannote) : les deux modes attribuent les propos.	Identification nominative des intervenants, multilingue, timestamps fins.
Classification	Un ton global de la réunion ou une liste de thèmes, via un appel LLM ou un modèle de sentiment pré-entraîné.	Classification par segment et multi-dimension (ton, thèmes, urgence).	Modèle dédié ou affiné, métriques de qualité, comparaison LLM vs modèle argumentée.
Compte-rendu + actions	Un résumé en texte libre généré par LLM.	CR structuré (décisions, actions avec responsable et échéance) en sortie structurée (JSON) rendue proprement.	Actions reliées aux responsables identifiés, statuts et relances, export PDF / e-mail / agenda.
Persistance + historique	Authentification simple ; chaque utilisateur retrouve ses réunions.	Modèle relationnel propre (réunions, segments, actions, thèmes) avec filtres.	Partage d'une réunion entre participants, rétention configurable, anonymisation effective.
Tableau de bord	Liste des réunions, accès au CR, un ou deux indicateurs simples.	Filtres (date, thème, statut) et graphes (thèmes dominants, actions ouvertes/fermées, temps de parole).	Tendances dans le temps, alertes d'actions en retard, croisements et export.
Architecture	Application propre, plutôt monolithique, traitement synchrone sur réunions courtes.	API (FastAPI) + front séparé, abstraction « source audio » commune aux deux modes, gestion des timeouts et erreurs API.	Traitement asynchrone (worker + file de tâches), webhooks de la plateforme visio, scalabilité pensée.
Tests & qualité	Quelques tests unitaires sur la logique non-IA, linting PEP8.	Couverture ≥ 70 % des fonctions critiques, appels API mockés dans les tests, stratégie documentée.	Tests d'intégration, test de charge (réunions simultanées), BDD/TDD assumé.

Brique	Socle (suffit à valider)	Cible (attendu)	Avancé (départage)
CI/CD & déploiement	Déploiement manuel reproductible (README + venv/Docker), application en ligne.	Pipeline CI (lint + tests sur PR), déploiement semi-automatisé (Railway, Render, Vercel).	CD complet, environnements séparés, monitoring, logs et alerting.
Benchmark (pré-prod)	Tableau comparatif honnête sur documentation publique : coût, quota, langues, RGPD, par famille d'API.	Mesures réelles de latence et de qualité sur quelques échantillons, via de petits scripts.	Protocole de mesure reproductible, KPIs chiffrés, recommandation argumentée avec sensibilité au volume.
RGPD & éthique	Données personnelles identifiées, consentement mentionné, durée de conservation définie.	Écran de consentement dans la maquette, analyse de risques, registre simplifié.	Consentement effectif en production, anonymisation des transcriptions, droit à l'effacement implémenté, DPA pour les sous-traitants API.

6. Livrables et exigences

6.1 Dossier de cadrage · groupe (PDF ≤ 15 pages)

1. **Vision produit** (1 p.) : elevator pitch + critères de succès.
2. **Personae & parcours** : 2 personae minimum (par exemple un manager en télétravail et un chef de projet en présentiel), journey-map illustrée pour chacun des deux modes de captation.
3. **Carte des user-stories** (User Story Map) avec priorisation MoSCoW. Les stories doivent être rattachées explicitement aux paliers (socle / cible / avancé).
4. **État de l'art / benchmark** sur quatre familles d'API : * ≥ 3 plateformes de **visioconférence intégrable** (par ex. Jitsi, LiveKit, Daily, Whereby, Twilio Video) ; * ≥ 3 API de **transcription avec diarisation** (par ex. Whisper + pyannote, AssemblyAI, Deepgram, Google Speech-to-Text) ; * ≥ 2 **LLM** pour le résumé et l'extraction d'actions ; * ≥ 1 approche de **classification** (LLM ou modèle NLP dédié). * Tableau comparatif : coût (\$), temps de réponse, quota, langues, conformité RGPD, capacité à récupérer l'audio.
5. **Analyse RGPD & éthique IA** : un enregistrement de réunion contient des voix (donnée biométrique), des propos identifiables et potentiellement confidentiels. Traitez le consentement des participants, la durée de conservation, le droit à l'effacement, et la question du DPA pour les sous-traitants. Indiquez la mesure correspondant à chaque palier.
6. **Roadmap de haut niveau** : release plan + backlog Sprint #1 détaillé (stories découpées et estimées). Le backlog doit garantir un socle livrable avant toute story de palier supérieur.
7. **Plan de risques** (≥ 8 risques, matrice probabilité/impact, mesures). Incluez explicitement les risques propres au sujet : dépassement du budget API sur des audios longs, latence de transcription, échec de l'intégration visio, qualité de diarisation insuffisante.
8. **Plan qualité** : métriques cibles, revue de code, seuil de couverture de tests.

6.2 Spécifications & architecture · groupe (PDF ≤ 10 pages)

1. **Diagrammes C4** : *Context* + *Container* complets. Le diagramme Container doit faire apparaître l'abstraction « source audio » commune aux deux modes de captation.
2. **Modèle de données** : ERD + dictionnaire d'attributs (utilisateur, réunion, segment de transcription, locuteur, action, thème).
3. **Séquence clé** : traitement complet d'une réunion (UML), depuis la captation jusqu'au compte-rendu, en distinguant le chemin visio et le chemin dictaphone.
4. **Choix d'API** : sélection argumentée à partir du benchmark, par famille d'API et par palier visé.
5. **Exigences fonctionnelles** : performance, SLO, budget API, durée maximale de réunion testée.
6. **Justification technologique personnelle** : choix des libs Python, pipeline CI/CD envisagé, monitoring.

6.3 Maquette interactive Streamlit · groupe

La maquette fige le parcours et sécurise les choix ; elle n'a pas vocation à faire tourner la chaîne réelle.

- Présentation des **deux parcours** : mode visio et mode dictaphone.
- **Mode dictaphone** : capture audio réelle possible via `st.audio_input`, suivie d'une transcription **mockée** (texte d'exemple).

- **Mode visio** : présenté au stade du **wireframe et du flow diagram**. Streamlit gère mal la visio en direct ; n'essayez pas d'y intégrer un SDK. La valeur de la maquette est de montrer le parcours, pas de faire fonctionner la visio.
- **Compte-rendu** affiché en placeholder (ou un appel LLM unique sur une transcription d'exemple).
- **Sélecteur de ton/thème** simulé + affichage d'icône.
- Gestion d'erreurs et indicateurs de chargement.
- **Écran de consentement** des participants (au moins maqueté), pour matérialiser l'analyse RGPD.
- Vidéo démo (≤ 3 min) + README d'installation.

6.4 Annexes obligatoires

- Tableau de benchmark (coût / performance) en `benchmark.xlsx`.
- Scripts de test prototypes (`pytest`), notamment de mesure de latence des API.

7. Détails des tâches

Catégorie	Tâches détaillées
Analyse besoin	Interviews éclair (2 utilisateurs fictifs : un usage visio, un usage présentiel), matrice Kano, priorités MoSCoW rattachées aux paliers.
UX/UI	Wireframes basse fidélité (Figma) des deux parcours, charte minimaliste, flow diagram, écran de consentement.
Benchmark	Scripts Python de mesure de latence des API (visio, STT, LLM), analyse de coût sur une réunion de 5 à 10 min.
Data model	Modèle relationnel + schéma JSON d'une réunion stockée (segments, locuteurs, actions, thèmes).
Architecture	Architecture cible de l'application web, avec l'abstraction « source audio » commune aux deux modes.
Qualité & tests	Stratégie BDD/TDD, plan de test de charge (réunions simultanées), définition du seuil de couverture.
Gestion projet	Création des tickets / stories sur Jira ou GitHub Projects pour la phase de Production, organisés par palier.

8. Contraintes & recommandations

- **Technos proposées** : Python 3.12, Streamlit $\geq$ 1.35, Git ; diagrammes : draw.io ou PlantUML.
- **Normes de code** : PEP 8 + type-hinting ; docstrings Google style.
- **Budget API** : max 15 € sur la pré-production (clés fournies). Cadrez vos réunions de test sur 5 à 10 minutes maximum et utilisez les fichiers d'exemple fournis pour ne pas épuiser le budget en essais.
- **Les deux modes de captation (visio et dictaphone) sont obligatoires.** En pré-production, ils sont **maquettés** ; ils ne deviennent pleinement fonctionnels qu'en phase de Production.